

JavaScript Runtime, Event Loop, and Execution Order

Objective

Understand how JavaScript executes code behind the scenes — including the role of the call stack, event loop, microtasks, and macrotasks. Build mental clarity about asynchronous behavior that affects real-world user experiences.

What's Expected

- Write a clear explanation in your own words
- Focus on reasoning, not repeating theory

Task

Create a file named:

event-loop-advanced.md

Write an article-style explanation covering:

- What is the JavaScript call stack and how it processes synchronous code
- What are microtasks and macrotasks, and how they are scheduled
- Why Promises, `queueMicrotask()`, and `setTimeout()` behave differently in execution order
- How the event loop decides what runs when

Exercise 1 – Microtasks vs Macrotasks

```
console.log('1');  
  
setTimeout(() => {
```

```

    console.log('2');
  }, 0);

Promise.resolve().then(() => {
  console.log('3');
});

queueMicrotask(() => {
  console.log('4');
});

console.log('5');

```

What to Do:

- Predict the **output order**
- Explain **why** each line executes when it does
- Identify which parts are:
 - Synchronous
 - Microtasks
 - Macrotasks
- Describe how the **event loop** handles each of them

Exercise 2 – Nested and Sequential Tasks

```

console.log('start');

setTimeout(() => {
  console.log('setTimeout 1');
  Promise.resolve().then(() => console.log('Promise 1'));
}, 0);

setTimeout(() => {
  console.log('setTimeout 2');
}, 0);

Promise.resolve().then(() => console.log('Promise 2'));

console.log('end');

```

What to Do:

- Predict the **output order**
- Explain your reasoning **step by step**
- Explain what happens during each **event loop tick**
- Identify which parts are:
 - Synchronous
 - Microtasks
 - Macrotasks

Key Takeaways

At the bottom of your file, write 3 bullet points summarizing what you learned about:

- Execution order
- Async behavior
- How it affects real projects